『大括号树』

改编 By ixRic/Lucky_Glass

数据 By ixRic

题解标程 By Lucky_Glass

引入

- 任何一道题都不是直接就得到最优算法的
- 这道题是从一开始的 $O(n^2)$ DP 找到灵感，然后扩展到能够处理树链然后才得到正解的
- 所以就先从部分分讲起

部分分 1

- 先 $O(n^2)$ 枚举一条链，但是注意链的方向是有影响的，即 u 到 v 的链不同于 v 到 u 的链。
- 然后就可以看成是“给你一个括号串，求其中有多少个正则匹配子串”。
- 最暴力的做法就是枚举子串的左右端点并判断是否正则匹配了。
- 可以先枚举左端点，右端点从左端点开始依次右移，可以维护当前是否正则匹配。所以是不需要额外遍历一遍子串，也就是一个 $O(n^4)$ 的算法。

部分分 2

- 从部分分 1 优化而来，鉴于大家都做过（应该吧？）原题括号树，做一个 $O(n)$ 的 DP 求括号串中正则匹配的子串的数量应该不难。
- 具体地说，f_i 表示以 i 结尾的正则匹配子串数量，在 DP 的时候从小到大枚举 i，用栈同时维护括号的匹配关系。
- 当 i 是‘)’，在栈中找到对应的‘(’假设为 j（可能没有），然后就有转移式

$$f_i = f_{j-1} + 1$$

- 对所有 f_i 求和即可。

部分分3

- 其实我觉得这个部分分还要暴力一些……但是的确对想出正解有帮助。
- 考虑一个正则匹配链(u,v)的贡献，就是 $siz_u \times siz_v$

部分分3

- 我们可以直接枚举一个 u，然后DFS找到一个能构成正则匹配链的v，此时 siz_u, siz_v 都很容易统计，于是复杂度 $O(n^2)$
- 中途可以剪枝，因为是 DFS
- 我就很担心会剪枝卡过……之前wzy随机生成了几个数据，还好 $O(n^2)$ 卡不过，然后wzy又加了一种完全二叉树的数据，只有叶子是)，其他全是 (，应该还是非常强力的。

部分分4

- 部分分3的计算贡献给了我们不少启发
- 这个部分分可以做到 $O(n)$（比正解还优秀 awa）
- 与之前 $O(n)$ DP 不同，我们不能单单只算子串的数量，还要算贡献；
- 定义 f_i 表示**只考虑从起始位置到 i** 的这条链，以 i 结尾的正则匹配子串的贡献。
- 我们看看这个贡献会怎么计算：

部分分4

- 于是就可以设计如下的DP

$$f_i = \sum_{j}^{(i,j)} j$$

$$f_i = f_{mat_i - 1} + mat_i$$

- mat_i是指 (i,j) 是正则匹配串的最大的 j
- 因为 f_i 只考虑了从起始到 i 的链的贡献，还要算上 i 后面，起始也简单，就相当于乘上 i 后面的大小

$$ans = \sum_{i} f_i \times (n - i + 1)$$

部分分小结

- 希望不要被部分分1,2带偏了……
- 部分分3告诉我们如何计算贡献
- 部分分4告诉我们如何将贡献用DP维护

- 接下来就需要把链搬到树上，怎么搬是个问题……
- 我选的是 DSU on Tree（树上启发式合并），当然你淀粉质应该也可以
- 接下来将会大篇幅写正解**如何实现**

正解

- 一个子树内的贡献分为两种：子树到根的链、子树到子树的链
- 考虑维护DP数组：
- $g_{u,i}$ 记录 u 子树内，从 u 到 u 的某一后继 v 的这条链上，右括号比左括号多 i 个并且后缀右括号不小于左括号的 v 的 siz_v 之和
- $f_{u,i}$ 记录 u 子树内，从 u 的后继 v 到 u 的这条链上，左括号比右括号多 i 个并且前缀左括号不小于右括号的 v 的 siz_v 之和

正解

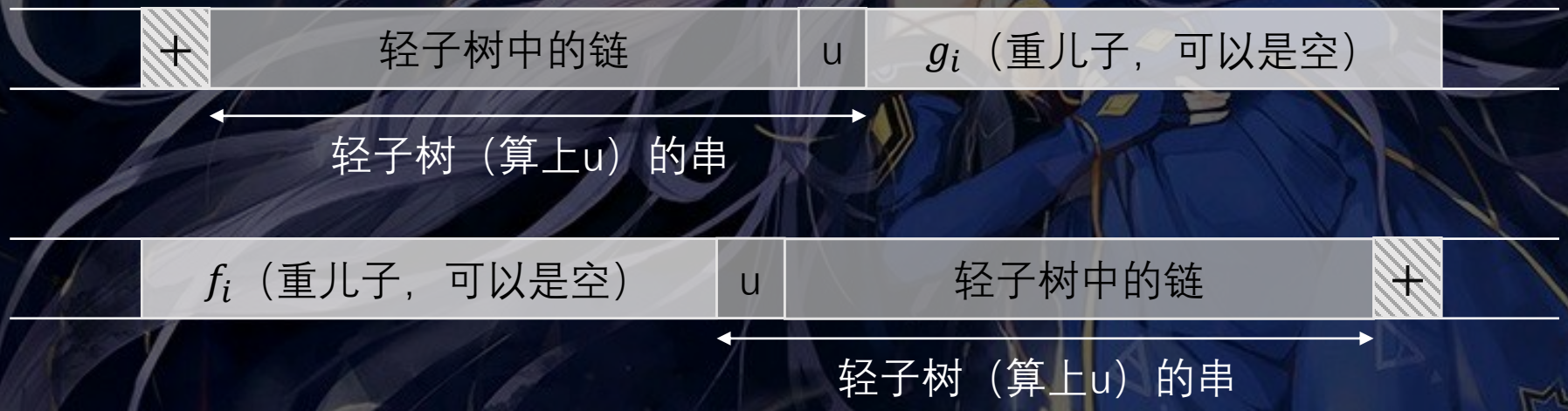
- 用图形表达就是这样：
- $g_{u,i}$ 维护的是一个潜在的能够匹配的后缀，需要补充一个前缀：

需要找到一个多余 i 个 (的前缀	剩余 i 个) 未匹配

- $f_{u,i}$ 维护的是一个潜在的能够匹配的前缀，需要补充后缀，差不多一样，就跟 $g_{u,i}$ 反过来。

正解

- 我写的是把 u 算到轻儿子里面去，所以先用重儿子已经计算出来的 f,g 去计算其他轻儿子的贡献；注意这样写就已经统计了u作为端点、轻子树作为另一端的情况；但是没有统计重子树和u作为端点的情况
- 大概是这样的：

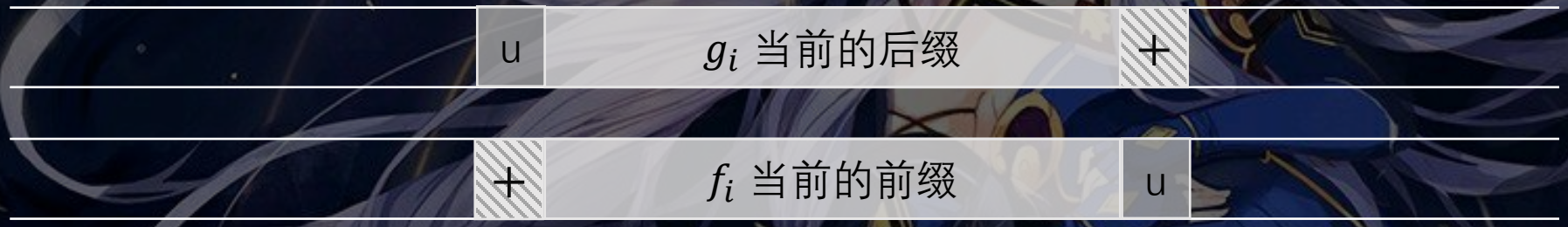


正解

- DFS轻子树时，维护轻子树的链上的前缀/后缀最小值——前缀时把‘(’看成1，后缀时把‘)’看成1（另一个是 -1）
- 要求前缀的每个前缀和都非负、后缀的每个后缀和都非负
- 前缀添加字符是在前端添加，后缀添加字符是在后端添加，因此维护前缀和后缀和都要线段树
- 这就是算法复杂度的瓶颈，最终复杂度 $O(\mathrm{n}\log^2 n)$，一个 $O(n\log n)$ 是DSU on Tree，再加上线段树
- 如果满足了前缀和非负，此时‘(’比‘)’多 i，则对答案贡献 $g_i \times siz_v$；后缀同理。

正解

- 接下来要把轻子树合并到重儿子上
- 也是一样的，只不过现在后缀是在后端加字符，前缀在前端加字符，还是要线段树



- 注意此时 u 是不算入前缀后缀的

正解

- 按照 DSU on Tree 的思想，我们先把重儿子 v 的 DP 数组和 u 的 DP 数组合并
- 若 u 是 ‘)’

$$f_{u,i-1} = f_{v,i}, g_{u,i+1} = g_{v,i}$$

- u 是 ‘(’ 差不多的道理
- 首先不可能 $O(siz_v)$ 遍历 DP 数组（会退化为 $O(n^2)$），其次DP数组开不下，只能边DFS边维护当前点的DP数组

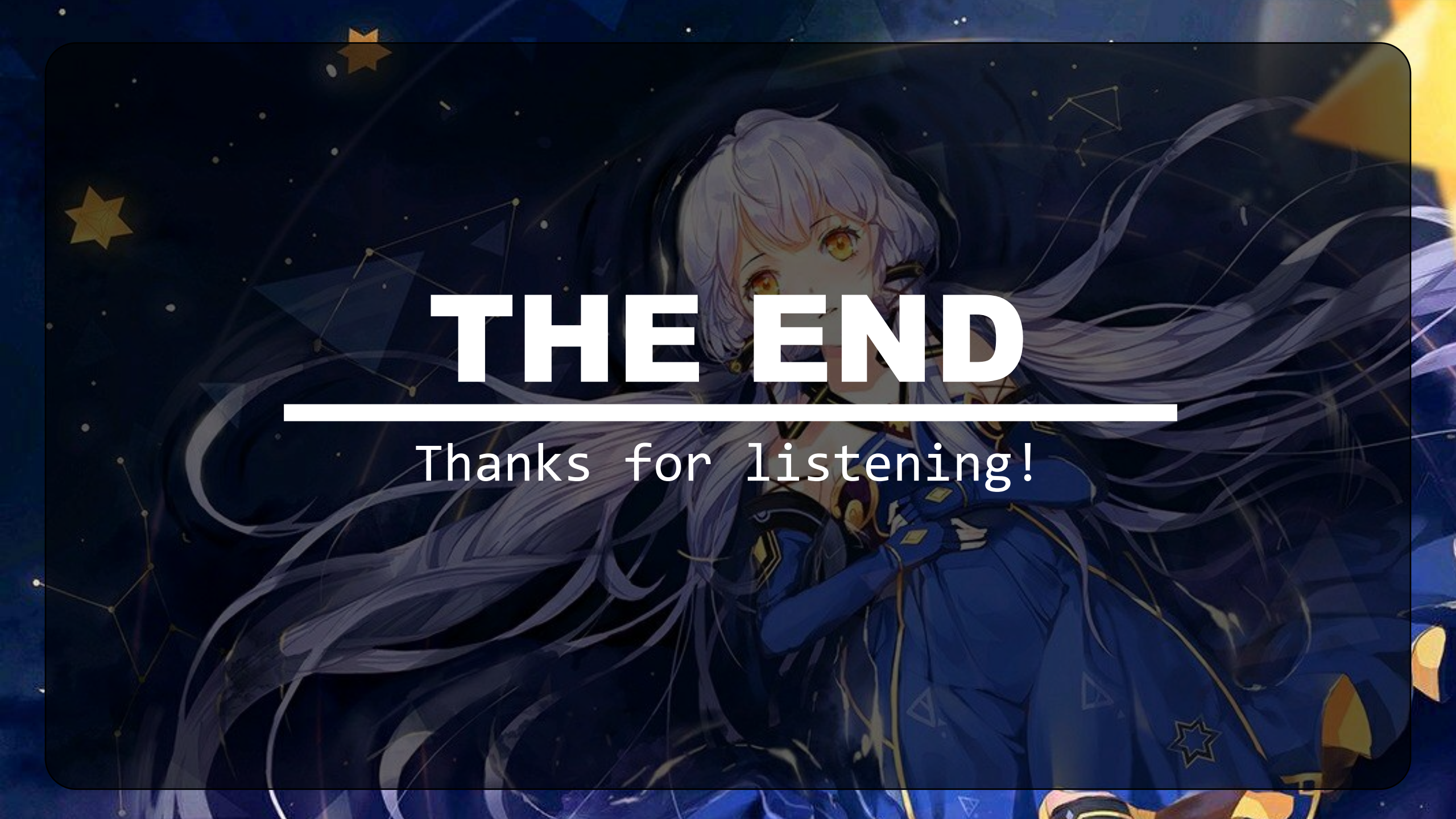
正解

- 我们发现上面其实是一个数组移位的操作
- ？？？
- 这个怎么实现?
- 我们直接把 f,g 数组开到两倍空间，然后分别用一个指针指向“实际零点”，可以用C++的指针，写起来也比较快乐

```
int aryf[2*N],*f=aryf+N;
f[0]=0,f++;//f[0]=0 是为了避免不合法位置没有清空
```

正解

- 综上，复杂度 $O(n\log^2 n)$ 可以通过。



THE END

Thanks for listening!